

Audit Report

Table of Contents

1. Executive Summary	5
About LoopScale	5
Audit Summary	5
Risk Profile	5
Overall Security Posture	5
Before Fix Review	5
After Fix Review	6
Launch Recommendations	6
Before Fix Review	6
After Fix Review	7
Audit Scope	7
2. Assumptions and Considerations	8
Assumptions & Considerations	8
Audit Assumptions	8
Trust Assumptions	8
3. Severity Definitions	9
Impact	9
Likelihood	9
Severity Classification Matrix	9
4. Findings	11
H01: update_single_ltv_lqt_entry Only Updates Uninitialized Entries	11
H02: Post-filter enumeration re-indexes active items sequentially, which can corrupt matrix updates.	12
H03: Inverted Logic When Deciding If a Loan Should Be Closed After a Collateral Withdrawal	13
H04: Users could alter the APY of a ledger by borrowing at 0% APY and instantly repaying	14
H05: Users could borrow principal at 0% APY	15
M01: Inconsistent Health Check Types Between Raydium and Orca Protocols	16

M02: Hardcoded Index 0 Breaking Orca Transfers at Other Collateral Slots	17
M03: Hardcoded Index 0 Breaking Raydium Transfers at Other Collateral Slots	18
M04: Incorrect Asset Identifier Constraint Blocking Position Transfers	19
M05: Incorrect Price Ratio Calculations in Delta Amount Functions	20
M06: Same protocol_position Used for Old and New Positions in transfer_position	21
M07: Incorrect account passed in OpenPositionWithToken22Nft in transfer_position	22
L01: Switchboard Integration Should Validate Against Negative Prices	23
L02: Missing auth check in the manage liquidity account struct.	24
L03: ALP oracle integration should not read price from paused pools	25
L04: Checking Cap Counter on CapOpType::Subtract Can Temporarily DoS Users Operations if Max Caps Are Updated	26
L05: Consider Adding an Oracle Fallback Mechanism	27
L06: Failure to Collect Rewards Before Closing Position	28
L07: Missing TickArrayBitmapExtension Support (Failure in Large Pools)	29
L08: Switchboard Price Feeds Low Samples Count Makes Price Vulnerable to Manipulation	30
L09: Current Use of Standard Deviation for Price Boundary Estimation Is Flawed	31

5. Enhancement Opportunities

32

E01: Additional Security Consideration for Switchboard Feeds	32
E02: Ambiguous Error Codes in Raydium Liquidity Management Operations	33
E03: Dead Code due to if statement guaranteed to not being executed	34
E04: Over-Updating Allocations in Multi-Collateral Borrows/Repays	35
E05: Some Tests Fail on Borrow Caps branch	36
E06: get_alp_price_with_quantity assumes the quote_mint is USD_QUOTE_MINT	37
E07: Max collateral allocation cap check is skipped when NAV is zero	38
E08: Dust Transfer Uses Saturating Sub	39
E09: Sanity checking input before forwarding to Raydium external programs	40
E10: Inconsistent handling of initial balances between handlers	41

About Us

42

About Adevar Labs	42
Audit Methodology	42
1. Program Context and Architecture Analysis	42
2. Threat Modeling	42
3. In-depth Manual Security Review	43
4. Detailed Fix Review and Validation	43
Confidentiality Notice	43
Legal Disclaimer	43
Appendix A: Switchboard Integration Statistics	45
Core Concern	45
Mathematical Inconsistency: Median + Standard Deviation	45
Robustness of Median vs. Mean	45
Scenario 1: Normal Oracle Submissions	45
Scenario 2: One Outlier	45
Scenario 3: Additional Submission	46
Invalid Distributional Assumptions	46
Alternative Approaches	46
Min and Max Price	46
Uncertainty Threshold	46

1. Executive Summary

About Loopscale

The Loopscale protocol is an order book-based lending platform on Solana.

The system uses direct order book matching instead of pooled liquidity to improve capital efficiency and risk management, featuring isolated markets, fixed-rate terms, and specialized primitives like Vaults and Loops for enhanced user experience.

Audit Summary

- Number of Findings: 21 total
- Critical: 0
- High: 5
- Medium: 7
- Low: 9
- Number of Enhancement Opportunities: 10

Risk Profile

The following table summarizes the distribution of identified vulnerabilities by risk level:

Risk Level	Count	Fixed	Acknowledged
Critical	0	-	-
High	5	5	0
Medium	7	7	0
Low	9	6	3

Overall Security Posture

Before Fix Review

The Loopscale team has demonstrated a solid understanding and experience in DeFi through their ability to architect and implement complex lending mechanisms. Most of the core instructions show good implementation, laying a positive foundation.

In addition to areas where logic patterns could be improved for consistency and accuracy, our review also uncovered a few significant issues.

Finally, while the current test suite provides a good foundation, enhancing its coverage, particularly for newer features and edge cases, would be beneficial.

Three specific areas warrant additional focus to further strengthen the protocol:

- **The recently introduced loan-lock feature:** This innovative addition, combined with the borrow-caps feature, expands the protocol's capabilities. Careful attention is needed to ensure these mechanisms interact seamlessly and don't inadvertently create unintended arbitrage opportunities.
- **Switchboard oracle integration:** Validating the feed parameters and statistical calculations within the Switchboard oracle integration is important to maintain the integrity of price feeds.
- **Raydium integration component:** This component showed the highest concentration of areas for improvement and would greatly benefit from thorough testing before deployment to ensure its robustness.

After Fix Review

All High and Medium severity issues have been comprehensively resolved, demonstrating the team's commitment to security and their technical proficiency in implementing fixes.

- **loan-lock feature:** The loan-lock feature has been fixed and deployed successfully.
- **Switchboard oracle integration:** The oracle integration now implements proper statistical validation and includes the most important safety checks. The minimum number of submissions per feed is not checked internally, this responsibility will rely on the market operator when choosing a price feed.
- **Raydium integration component:** Previously the area of highest concern, the Raydium integration has undergone comprehensive fixes.

Launch Recommendations

Before Fix Review

To ensure a smooth and secure launch, we offer the following recommendations:

I. Mandatory Before Launch:

- Ensure all "High" and "Medium" issues are fixed
- Carefully review and test the implications of the loan-lock feature with newly added mechanisms
- Ensure the presence of the **bs_auth** Signer on every sensitive instruction

II. Strongly Recommended:

- Conduct comprehensive testing of the Raydium integration, including reward collection, position transfers, and tick array bitmap support.
- Consider reviewing the oracle architecture to explore opportunities for enhanced resilience against adversarial events.
- Review and compare the use of saturating arithmetic, especially in dust calculations where loss would be unacceptable
- Consider integrating additional invariant checks directly within functions

III. Post-Launch Monitoring:

- Deploy continuous fuzzing in the CI/CD pipeline
- Continuously monitor Switchboard feed health
- Track oracle manipulation attempts and unusual price movements that could indicate attacks.

After Fix Review

- "High" and "Medium" issues have been fixed.
- The **bs_auth** Signer has been implemented on missing instructions.
- Oracle architecture resilience has been significantly improved with robust validation against invalid prices and exceptional market events. We continue to recommend implementing a fallback oracle mechanism for cases where the primary market oracle becomes unresponsive.
- Arithmetic operations have been updated to use checked subtraction where precision is critical, replacing potentially unsafe saturating arithmetic in dust calculations

All other recommendations represent ongoing security best practices that should be maintained post-launch through continuous monitoring and iterative improvements.

Audit Scope

Branch feat/loan-lock

- **Repository:** <https://github.com/LoopscaleLabs/loopscale-program-library>
- **Files/Modules in Scope:**
- PR [#1122](#)

Branch feat/borrow-caps

- **Repository:** <https://github.com/LoopscaleLabs/loopscale-program-library>
- **Files/Modules in Scope:**
- PR [#1087](#)

**Branch feat/raydium-clmm

- **Repository:** <https://github.com/LoopscaleLabs/loopscale-program-library>
- **Files/Modules in Scope:**
- PR [#1108](#)

Branch feat/switchboard

- **Repository:** <https://github.com/LoopscaleLabs/loopscale-program-library>
- **Files/Modules in Scope:**
- PR [#1154](#)

Branch feat/apl

- **Repository:** <https://github.com/LoopscaleLabs/loopscale-program-library>
- **Files/Modules in Scope:**
- PR [#1210](#)

2. Assumptions and Considerations

Assumptions & Considerations

Audit Assumptions

The following limitations and constraints should be considered when reviewing this security assessment report:

I. Refinance and Liquidate do not count towards Supply and Withdraw Cap

RefinanceLedger and **LiquidateLedger** instructions do not count towards the supply and withdraw caps.

II. Principal Counters fully reset at the end of the cap's interval

The **principal_*** counters reset to 0 at the end of each interval (eg. hourly), allowing potential cap exceedance. For instance, a 1-hour withdraw cap can effectively reach 2x the limit: full cap usage in the last minute of one interval, plus another full cap immediately after reset. The naming is misleading, as it doesn't reflect net principal but gross flows.

III. Deposit and Withdraw Caps account for the overall net capital value

In the feat/borrow-caps branch, deposit and withdraw caps (eg. per hour, per day) have been introduced to limit user actions. However, the implementation updates counters in an unexpected manner: during a deposit, the deposit counter is incremented (as expected), but the withdraw counter is decremented. Conversely, during a withdraw, the withdraw counter is incremented, but the deposit counter is decremented.

This behavior enables scenarios where caps can be effectively bypassed without altering the net asset delta. For instance, coordinated large deposits and withdrawals could result in both deposited and withdrawn amounts exceeding their respective caps, even if the overall balance remains unchanged (eg. one user deposits heavily while another withdraws equivalently). This could lead to unintended overuse of the system, potential liquidity risks, or exploitation in high-volume environments.

IV. Borrow and Repay Cap Tracking Inflates Per-Asset Allocations

The borrow and repay operations incorrectly track collateral asset allocations by adding/subtracting the full operation amount to each collateral asset's cap tracking, rather than splitting the amount proportionally across assets. This implementation flaw inflates the per-asset allocation counters by a factor equal to the number of collateral assets involved in the operation.

Trust Assumptions

- Market parameters can be updated by admins, but are subject to a configurable timelock
- No fallback oracles, implying that oracles are expected to always respond

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Impact

Impact reflects the potential consequences of the issue—particularly on **project funds, user funds,** and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

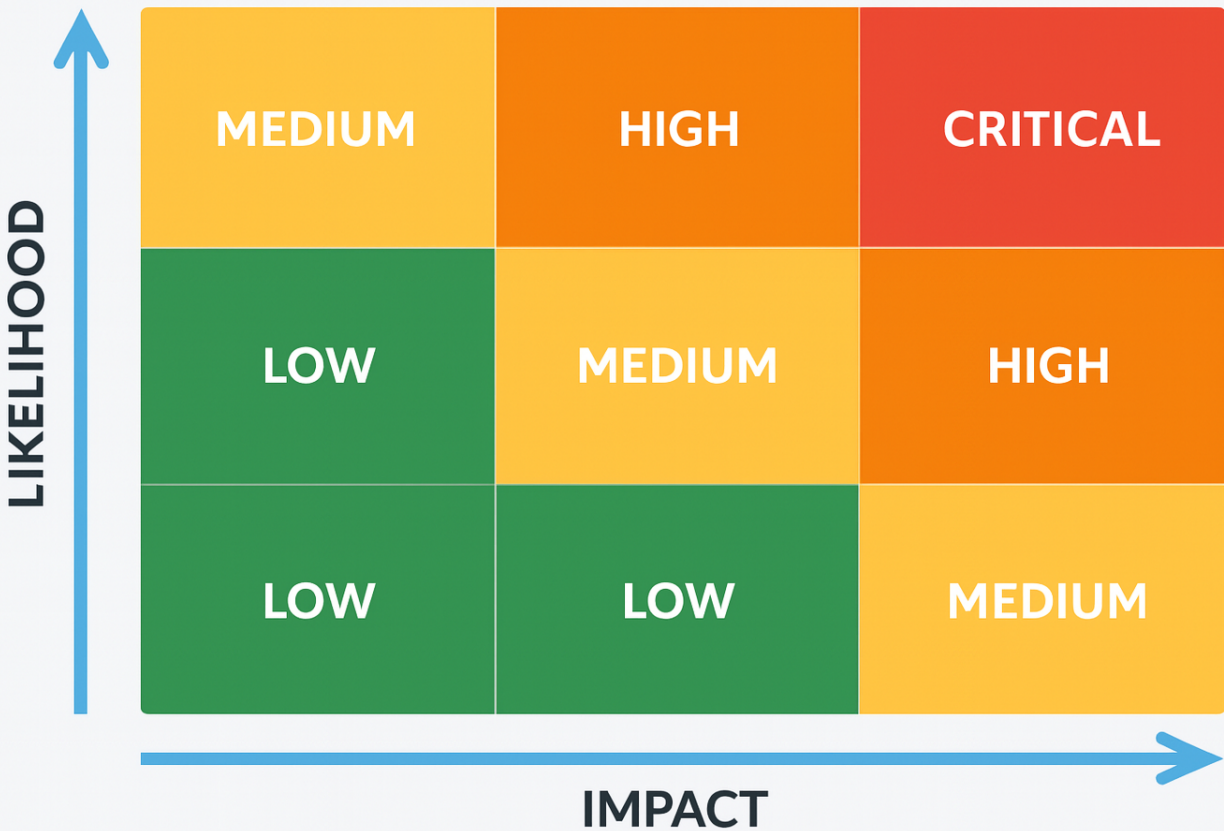
- **High Likelihood:** The vulnerability is trivially exploitable, and/or it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements, and/or low financial risks.
- **Medium Likelihood:** Can be found and exploited with moderate effort, and could require a significant capital investment with manageable financial risk.
- **Low Likelihood:** Exploitation may be impractical or theoretical due to technical constraints and could require a significant financial risk for the attacker, with a high risk of loss relative to potential gains.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substansial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability
- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification

- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

Appendix A: Switchboard Integration Statistics

Core Concern

Our analysis identified several issues in the integration of Switchboard oracle feeds. Primarily, we observed an incorrect application of standard deviation to the median for confidence interval calculations. This issue is critically linked to the number of price samples available at each timestamp, suggesting that the price feeds could be highly susceptible to price manipulation.

Mathematical Inconsistency: Median + Standard Deviation

The current implementation utilizes the **median** as the base price but calculates confidence intervals using the **standard deviation**. This approach is mathematically inconsistent for the following reasons:

- **Standard deviation** measures the spread around the **mean**, not the median.
- These statistics belong to different mathematical frameworks: the median is a robust statistic resistant to outliers, whereas standard deviation assumes a mean-centered distribution.
- Consequently, the resulting confidence intervals lack a sound mathematical basis and may provide a misleading sense of security regarding price bounds.

Robustness of Median vs. Mean

Let's examine a realistic oracle scenario with five ETH price submissions (in USD):

Scenario 1: Normal Oracle Submissions

Submissions: [2000, 2005, 2015, 2020, 2025]

- **Mean** = $(2000 + 2005 + 2015 + 2020 + 2025) / 5 = 2,013$
- **Median** = 2015 (middle value)
- **Standard Deviation** = 9.27

In this scenario, the median and mean are very close, both accurately representing the true price. While applying standard deviation to the median is mathematically incorrect, the error introduced might be negligible if both values are sufficiently close.

Scenario 2: One Outlier

Submissions: [2000, 2005, 2015, 2020, 2200] (Attacker submits an extreme value: 2025 is replaced by 2200)

- **Mean** = $(2000 + 2005 + 2015 + 2020 + 2200) / 5 = 2048$ (Shifted by approximately 2%)
- **Median** = 2015 (Still the middle value, demonstrating robustness)
- **Standard Deviation** = 84.3 (Massively inflated)

While the median successfully identifies the true market price of 2015 despite the manipulation attempt, the standard deviation calculation is corrupted by the same outlier that the median

disregarded. This creates a fundamental inconsistency: the price estimate is accurate, but the uncertainty bounds are rendered meaningless.

Scenario 3: Additional Submission

Let's consider a specific scenario where the median could be more susceptible to manipulation:

Submissions: [1995, 2000, 2005, 2015, 2020, 2025] (Attacker submits an additional value)

- **Mean** = $(1995 + 2000 + 2005 + 2015 + 2020 + 2025) / 6 = 2010$ (Shifted by approximately 0.1%)
- **Median** = 2007.5 (Shifted by approximately 0.5%)
- **Standard Deviation** = 10.80

This scenario reveals that the median is not universally more robust than the mean. When an attacker can add new submissions rather than replace existing ones, the median's vulnerability depends on the sample size and distribution. With small, even-numbered samples, strategic placement of outliers can shift the median more significantly than the mean.

Invalid Distributional Assumptions

The confidence interval calculation uses a **1.96 multiplier** (for 95% confidence), which implicitly assumes:

- Oracle price submissions follow a **normal distribution**.
- Sample sizes are sufficient for the **Central Limit Theorem** to apply.
- Submissions are **independent and identically distributed**.

The typical threshold for normal distribution assumptions to be valid is $n \geq 30$. For feeds with a sample size below this value, the use of confidence intervals loses statistical meaning.

Alternative Approaches

Min and Max Price

The Switchboard feeds return a **submissions** field, which contains all individual price submissions. An alternative approach for defining price bounds would be to use the submissions at positions **n-1** and **n+1**

from the median value. This approach is more conservative than relying on the **min_value** and **max_value** fields, which might represent values too far from the true price.

Uncertainty Threshold

Given that standard deviation is not suitable for small sample sizes, alternative approaches can help detect suspicious activity on the price feed:

- The Median Absolute Deviation (MAD) is a measure of statistical dispersion that is resilient to outliers.
- Calculate the relative spread around the median using the **n-1** and **n+1** submissions. For example: **Relative Spread** = $(\text{upper_bound} - \text{lower_bound}) / \text{median} * 100$.

These values can then be compared against a threshold, which should be determined based on the volatility of the analyzed asset:

1. **Stablecoins:** Characterized by very stable prices.
2. **BTC, ETH, Altcoins:** Generally less volatile than memecoins during periods of high market movement.
3. **Others (e.g., Memecoins):** Exhibit very high volatility.